

Aula 1. Visão geral da teoria de sistemas

Objetivos:

- identificar a origem da teoria geral dos sistemas, bem como os conceitos de dado, informação e sistema de informação; e
- reconhecer o que é análise de sistemas e o que faz o analista de sistemas.

Caro(a) estudante,

Bem-vindo(a) à nossa primeira aula! Espero que possa percorrer o caminho da nossa disciplina da melhor maneira possível. Passaremos juntos boa parte da caminhada de sua formação. Espero contribuir para a sua aprendizagem. Bons estudos.



1.1 Introdução

Nos dias de hoje, a informação, sob o impacto da utilização de tecnologia, tem uma influência cada vez maior no mundo dos negócios. A empresa que dispõe de mais informações sobre seu processo de negócio está em vantagem competitiva. Isso coloca a informação como o recurso-chave de competitividade efetiva e de diferencial de mercado.

A informação também é considerada e utilizada como um fator que instrumentaliza e estrutura a tomada de decisão dentro de uma organização. Dessa forma, é importante que sua leitura seja feita de forma mais precisa possível, para que os tomadores de decisão não possam vir a incorrer em erros.

Na criação de um sistema de informação, um dos seus componentes é o sistema de software, compreendido de seus módulos funcionais para automatizar e controlar as diversas tarefas.

Antes de tudo, vamos entender o que é um sistema?

1.2 Teoria geral dos sistemas

Um sistema é definido como um complexo de elementos em interação, sendo esta de natureza ordenada, gerando uma unidade.

Então, de acordo com a teoria geral dos sistemas:



Sistema é um conjunto de partes inter-relacionadas que se interagem para alcançar um objetivo comum.

Todo sistema é composto por subsistemas. Além de visualizá-lo e entendê-lo na sua totalidade, temos que observar e compreender as partes que se interagem para formá-lo.

Ex.: o corpo humano é um sistema formado de cabeça, tronco e membros (subsistemas).

Um sistema computacional de RH é composto pelos subsistemas de folha de pagamento, controle de frequência, gestão de pessoas etc.

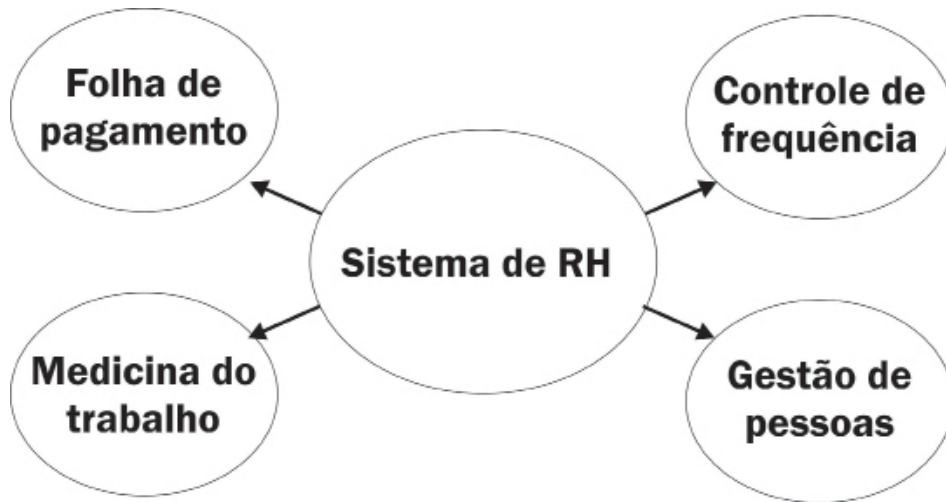


Figura 1. Exemplo de sistema

Fonte: autor

Sistemas físicos ou concretos: quando compostos de equipamentos, são denominados “hardware”.

Sistemas abstratos ou conceituais: quando compostos de conceitos, planos, hipóteses e ideias, são denominados “software”.

1.4 Conceitos de análise de sistemas e analista de sistemas

Vamos compreender o que é análise de sistemas:

Análise de sistemas é a atividade de realizar estudo dos processos, métodos e técnicas de investigação e especificação da solução do problema, a fim de, a partir dos requisitos levantados, encontrar o melhor caminho para a criação de software.

A análise é o processo de separação das partes de um sistema para facilitar o entendimento de sua natureza, funções e relações.

Podemos definir também análise de sistemas como o processo de analisar, projetar e implementar sistema de software.

E o analista de sistemas?

Analista de sistemas é o(a) profissional que define o que, para que e de que forma será desenvolvida uma solução executada pelo computador.

Resumo

Nesta primeira aula vimos que, nos dias de hoje, a informação, sob o impacto da utilização de tecnologia, tem uma influência cada vez maior no mundo dos negócios. Apresentei também o conceito de sistema (um conjunto de partes inter-relacionadas que se interagem para alcançar um objetivo comum) através da teoria geral dos sistemas.

Conceituei ainda dados, informações e sistemas de informações. Vimos que dado é qualquer elemento identificado em sua forma bruta que por si só não conduz a uma compreensão de determinado fato ou situação, que informação é o dado processado e que sistema de informação, por sua vez, é um conjunto de regras e procedimentos organizados para o fornecimento de informações para o usuário.

Por fim, apresentei-lhe que análise de sistemas é a atividade que consiste em realizar estudo dos processos, métodos e técnicas de investigação e especificação da solução do problema, com a finalidade de, a partir dos requisitos levantados, encontrar o melhor caminho para a criação de software, e que analista de sistemas é o(a) profissional que define o que, para que e de que forma será desenvolvida uma solução executada pelo computador.

Aula 2. Modelagem de sistema

Objetivos:

- reconhecer o conceito de modelo do sistema; e
- identificar o que é um sistema orientado a objeto.

1. Introdução

Para a criação de um sistema de informação, primeiramente, é essencial definir os objetivos a serem alcançados. Independentemente do método a ser adotado para a análise e desenvolvimento de um sistema, tudo se inicia com uma investigação das necessidades pelo cliente, seja ele interno (colaboradores dentro da mesma organização) e/ou externo. Assim, todo sistema de informação é concebido para resolver um ou mais problema(s).

É preciso esgotar toda discussão sobre as necessidades reais para solucionar o problema em questão, garantindo que o objetivo do sistema seja alcançado. E a complexidade do problema cresce à medida que cresce o tamanho do sistema. Para entender a complexidade do problema e definir um modelo ideal de solução, precisamos conhecer alguns conceitos da análise de sistema que vão ajudar a moldar a melhor solução para um problema em estudo.

2. Conceito de modelo de sistema

O modelo conceitual descreve a visão que o usuário tem das informações existentes que serão gerenciadas pelo sistema. Essas informações existentes

Assim como na construção do mobiliário completo, na construção de sistemas de software também é necessário um planejamento inicial, equivalente ao projeto do design do mobiliário. Isso nos leva ao conceito de modelo.

Modelo é uma representação do sistema a ser desenvolvido. Ex.: desenho da interface gráfica do sistema.

Por que construir modelo do sistema?

- Para ter um entendimento melhor da complexidade do sistema. Modelos descrevem as características relevantes de um sistema. Os detalhes não relevantes só aumentam a complexidade no entendimento do problema e podem ser ignorados aplicando o princípio da abstração.

Usamos o conceito de abstração quando nos concentramos nas coisas que consideramos essenciais e desprezamos as outras não relevantes dentro do contexto da análise. É a visão que obtemos da realidade do mundo real. Ex.: conceito de cadeira (assento, encosto) – abstração.

- Para difundir as informações relativas ao sistema entre os indivíduos envolvidos no projeto.
- Para gerenciar os custos no desenvolvimento do sistema. Erros identificados na fase de construção do modelo têm impacto menos desastroso.

Os modelos são representados por diagramas, que é a forma gráfica de expressar a ideia do sistema a ser desenvolvido.

2.3 Conceito de sistema orientado a objeto

Vamos voltar ao conceito de sistema, estudado na primeira aula, em que estabelecemos que um “sistema é um conjunto de partes inter-relacionadas que se interagem para alcançar um objetivo comum”.

Fazendo uma analogia, um sistema de software pode ser formado por objetos que se interagem através de mensagens para que uma tarefa computacional seja realizada.

Aplicada, a orientação a objetos na modelagem de sistemas de software faz com que os modelos construídos para solucionar um problema fiquem mais próximos do mundo real e estabelece que qualquer coisa é um objeto que pertence a uma classe, e esta é um repositório de informação associado ao objeto.

- Para que você, caro(a) estudante, compreenda melhor a construção dessa modelagem, vou exemplificar o conceito de classe, objeto e suas características.

2.3.1 Conceito de classe e objeto

Definimos como objeto todas as coisas do mundo real e que fazem parte de uma classe. Diz-se que um objeto é uma instância de uma classe.

Objeto é qualquer elemento do mundo real.

Ex.: bicicleta, professor, conta bancária, ícone etc. Atributo é um conjunto de

características de um objeto.

Ex.: nome de um livro, saldo bancário do cliente, modelo de um carro, cor da caneta, graduação do professor etc.

Classe ou classe de objeto é uma abstração das características relevantes de coisas do mundo real. É um conjunto de objetos que têm a mesma especificação.

Ex.1: Classe – aluno.

Objeto da classe - Pedro, Maria etc.

Atributos – matrícula, nome, endereço, notas, curso etc.

Ex.2: Classe – veículo.

Objeto da classe – ônibus, carro, moto etc.

Atributos - motor, marca, modelo etc.

2.3.2 Conceito de método e mensagem

Os métodos descrevem o comportamento do objeto, que são suas funcionalidades (funções, operações).

Os objetos interagem através do envio de mensagens. Isso significa que os objetos estão enviando mensagens uns aos outros com o objetivo de realizar alguma operação (requisitando serviço) no sistema.

Ex.: Classe: impressora.

Atributos: velocidade, resolução, processador. Métodos: ligar /desligar,

imprimirarquivo.

Ex.: o objeto **Relatório Cadastro de Fornecedor** envia mensagem **ImprimirArquivo** para o objeto impressora.

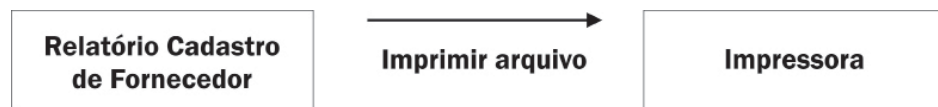


Figura 3. Representação gráfica de envio de mensagem entre objetos.

Fonte: autor



Ex.: Classe de objeto: aluno.

Atributos: nome, matrícula, nota, média do aluno.

Método: calcularmédia (operação).

3. Princípio da abstração

O princípio da abstração, dentro do conceito da análise orientada a objeto, define alguns mecanismos de análise para gerenciar a complexidade do problema e concentrar a atenção nas características essenciais de um objeto, como os mecanismos encapsulamento, polimorfismo e herança.

1. Encapsulamento

O mecanismo de encapsulamento restringe o acesso às operações internas realizadas por um objeto. Um objeto envia mensagem a outro objeto para realizar determinada tarefa, sem se preocupar com o modo como a tarefa é realizada. A aplicação desse mecanismo esconde os detalhes de funcionamento interno de um objeto. Nesse mecanismo, são disponibilizadas para o usuário apenas as funções que o objeto executa através da interface do sistema e encapsula a sua implementação, protegendo assim o acesso indiscriminado aos dados.

O acesso é feito através de mensagens (operações) que são trocadas entre objetos. A implementação do objeto pode ser modificada sem perder a sua funcionalidade e a maneira como é operada e conhecida pelo usuário.



Figura 4. Exemplo de encapsulamento.

Fonte: autor

Na figura acima, temos o objeto computador e o objeto data show, em que cada um tem suas funções. A integração entre eles é feita através da ligação da saída de um objeto à entrada do outro objeto. O data show pode reproduzir as imagens do computador. A implementação está encapsulada no próprio objeto. O usuário precisa saber apenas onde conectar a entrada de

um objeto e a saída do outro objeto para produzir a imagem.

2. Polimorfismo

No mecanismo denominado polimorfismo, um objeto pode enviar a mesma mensagem para objetos semelhantes, mas as interfaces são implementadas de formas diferentes.

Ex.:



Figura 5. Exemplo de polimorfismo. O objeto controle remoto envia mensagem (requisitando serviço) para o objeto DVD e o objeto televisão.

3. Generalização de abstração (herança)

O mecanismo conhecido por generalização de abstração (ou simplesmente herança) ocorre quando definimos um subconjunto de relacionamentos entre elementos de duas ou mais classes.

Ex.: classe veículo é a generalização da classe motocicleta, pois toda motocicleta é um veículo.

Por esse mecanismo podemos estabelecer relações entre as classes, compartilhando os atributos e operações semelhantes.

Ex.: Classe pessoa

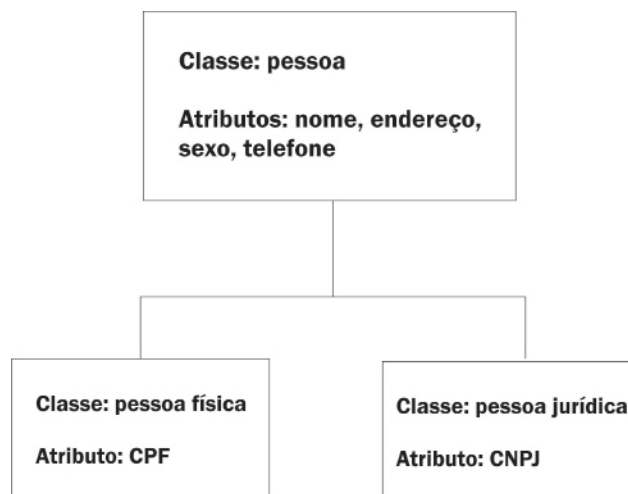


Figura 6. Exemplo de herança.



Aula 3. Processo de desenvolvimento de sistema

Objetivos:

- identificar as fases de análise de sistema; e
- construir um modelo ideal para a solução de um problema real.

1. Introdução

O principal objetivo do levantamento de requisitos para o desenvolvimento de um software é que o usuário (cliente) e o analista (desenvolvedor) tenham a mesma visão do problema a ser resolvido, e que juntos consigam definir as necessidades (os requisitos) para a solução do problema.

O levantamento de requisitos que irá compor o sistema é a etapa mais importante, porque é nessa fase que será definida a qualidade do sistema, que é medida pela usabilidade do software. E a usabilidade será medida se os requisitos atenderem às necessidades do usuário.

Um sistema de informação é utilizado para automatizar processos, e esses processos e seus requisitos devem ser compreendidos antes do desenvolvimento do sistema de informação.

2. Processo de desenvolvimento

O processo de desenvolvimento de software compreende as atividades de levantamento de requisitos, análise, projeto, implementação e teste.

3.2.1 Levantamento de requisitos

Na fase de levantamento de requisitos, trabalha-se para levantar, analisar, documentar e validar as necessidades do cliente ou de algum produto novo dentro de um projeto de software. Essas necessidades são inicialmente definidas como funcionalidades, ideias ou desejos de como o sistema deve funcionar, e normalmente são transformadas em requisitos funcionais e não funcionais do projeto.

Mas o que são requisitos?

Requisito é o que o sistema tem que ter para atender os objetivos para o qual o sistema é criado.

E o que é requisito funcional?

Definimos a funcionalidade a ser desenvolvida para automatizar uma necessidade do sistema.

Ex.: funcionalidade: existe uma necessidade de definir uma funcionalidade no sistema para gerar um relatório de alunos por período.

Requisito funcional: o sistema deve permitir a geração de um relatório com todas as informações de alunos ocorridas entre duas datas informadas pelo usuário.

E requisito não funcional?

São definidos como requisitos não funcionais aqueles que descrevem apenas atributos do sistema ou atributos do ambiente do sistema. São os requisitos que geralmente estão mais ligados ao uso do software.

Ex.: o usuário, para ter acesso ao módulo de relatório através da interface do sistema, precisa estar cadastrado e possuir um login e senha.

Uma maneira de reconhecer os requisitos do sistema é criar um questionário intuitivo para identificar as funcionalidades do sistema e produzir um documento texto chamado de sumário executivo, que deve descrever o que o(a) analista conseguiu pesquisar de relevante sobre o sistema em estudo e o que o cliente espera que o sistema faça.

Ex.: Questionário:

- Qual o nome da funcionalidade? Cadastro de aluno.
- Quais as informações que irão compor esse cadastro? Nome, endereço, telefone, CPF, RG, e-mail etc.
- Onde essas informações serão armazenadas? Banco de dados, acesso pela rede.

Com essas perguntas básicas, definimos alguns requisitos funcionais e não funcionais para criar um cadastro de aluno.

- Certamente, para identificar todos os requisitos de um sistema, é necessário um questionário mais detalhado.

Fazendo uma analogia, vou exemplificar um estudo de caso conhecido de todos: como realizar o “Churrascão” de domingo.

- Nome da funcionalidade: Churrascão de domingo.
- Quais os requisitos funcionais do Churrascão: carne, tempero, carvão, bebida, farofa, arroz.
- Quais os requisitos não funcionais do Churrascão: onde o evento (churrascão) irá acontecer.

Vou aplicar agora outra metodologia para especificar os requisitos, exemplificando um sumário executivo para um sistema em que as funcionalidades são bem conhecidas de todos.

Ex.: sumário executivo: sistema de biblioteca.

Nesse documento é proposto o desenvolvimento de um sistema de controle de uma biblioteca para informatizar as funções de empréstimo e devolução de livros. O sistema deverá gerar relatório mensal de empréstimos por livro e usuários cadastrados e calcular automaticamente os pagamentos de multas em caso de devolução fora do prazo estipulado pelo sistema. O sistema deverá, ainda, inabilitar o cadastro de usuários da biblioteca que não devolverem os livros emprestados.

Esse documento descreve as principais necessidades e funcionalidades do sistema a ser desenvolvido, e que serão mais bem estruturadas nas outras fases do desenvolvimento.

Podemos criar uma tabela para identificar os requisitos para o sistema pro- posto no sumário executivo.

Requisito funcional: emprestar livro	
Descrição: o sistema deve registrar empréstimos de livros, informando o usuário e o livro emprestado, a data de empréstimo e a data prevista de devolução.	
Requisitos não funcionais: controle de acesso, identificação dos livros e identificação de usuários, armazenamento dos dados	
Descrição:	
Controle de acesso	Esta função só pode ser acessada por usuário do sistema com perfil de administrador do sistema.
Identificação de livros	Os livros serão identificados por um código de barras.
Identificação do usuário	O usuário será identificado pelo seu CPF.
Armazenamento dos dados	Os dados serão armazenados em um banco de dados.

Requisito funcional: devolver livro	
Descrição: o sistema deve registrar a devolução de livros, checar a data de devolução, se não estiver dentro do prazo, calcular multa sobre os dias de atraso.	
Requisitos não funcionais: controle de acesso, identificação dos livros e identificação de usuários	
Descrição:	
Controle de acesso	Esta função só pode ser acessada por usuário do sistema com perfil de administrador do sistema.
Identificação de livros	Os livros serão identificados por um código de barras.
Identificação do usuário	O usuário será identificado pelo seu CPF.

Figura 8. Exemplo de tabelas que compõem os requisitos funcionais e não funcionais.

- A fase de levantamento de requisitos deve ser uma fase de descoberta, em que o analista, cliente e usuários vão identificando as funcionalidades e as restrições. Os requisitos não identificados nessa fase deverão ser realocados ao longo do restante do processo de desenvolvimento. É possível também que os requisitos mudem durante o processo de desenvolvimento, e deve-se gerenciar esse tipo de mudança para eventualmente realocar novos requisitos.

2. Análise

Na fase da análise faz-se a investigação do problema. Essa fase é importante para entender com clareza um problema de sistema de informação, e para isso, o problema deve ser bem investigado.

A análise é para produzir uma ampla compreensão do problema, identificando e avaliando as informações levantadas para o projeto de sistema.

3. Projeto

A fase do projeto propõe uma solução que atenda os requisitos levantados na fase da análise, baseado no conhecimento adquirido.

A fase do projeto trata da construção das especificações detalhadas para a construção do sistema.

Essas especificações incluem o projeto das interfaces, banco de dados, estrutura física e lógica, modelagem, processo, escopo, dicionário de dados e cronograma.

São utilizados nessa fase os diagramas da UML, tais como diagrama de caso de uso, diagrama de classe, diagrama de sequência etc., para a construção do projeto.

4. Implementação

Nesta fase o sistema é codificado, ou seja, ocorre a tradução dos requisitos especificados na fase do projeto em código executável através do uso das linguagens de programação, como Java, frameworks e estruturas de dados.

5. Testes

Nesta fase são realizados testes para verificar se o sistema construído atende às especificações detalhadas na fase do projeto e a geração de relatórios de testes, que contém informações sobre os possíveis erros detectados.

- O envolvimento do cliente usuário é importantíssimo no desenvolvimento de um software e para a satisfação do cliente.

Resumo

Nesta aula foram apresentadas algumas reflexões sobre a importância da fase de levantamento de requisitos e como são identificados os requisitos funcionais e não funcionais, através de uma tabela demonstrativa ou pelo sumário executivo, para modelar as informações que farão parte do projeto de desenvolvimento de um sistema de informação.

Atividades de aprendizagem

1. Tomando como base um cenário do mundo real, tente visualizar uma necessidade de controle das informações de uma escola perto de sua casa ou que você conheça. Procure conversar com alguém que tenha o conhecimento do trâmite das informações que devem ser controladas e, fundamentando-se nos exemplos estudados, faça um levantamento de requisitos para o desenvolvimento de um sistema escolar.

Responda o questionário proposto ou crie o seu questionário, baseado na conversa com o seu cliente, nesse caso a escola.

Faça uma explanação inicial ao dirigente da escola sobre o propósito das perguntas para que você, caro(a) estudante, possa criar uma base de conhecimento sobre as necessidades de controlar determinadas informações de uma escola para o desenvolvimento de um sistema de informação.

Questionário proposto:

a) Quais os objetivos do sistema?

b) Quais as atividades relacionadas ao sistema?

c) Quais os problemas referentes a cada atividade?





d) Quais as prováveis soluções para os problemas de cada atividade?

e) Quais as informações envolvidas nas soluções?

f) Quais as decisões associadas à atividade?

g) Quais os produtos, bens ou serviços produzidos por cada atividade?

2. Escreva o sumário executivo para o sistema proposto.

3. Descreva os requisitos funcionais e não funcionais para o sistema proposto.



Dessa maneira terminamos a nossa terceira aula. Já estamos quase na metade da nossa caminhada. Vamos prosseguir? Encontramo-nos na quarta aula. Até!



Aula 4. Metodologia para desenvolvimento de projeto de software

Objetivo:

- identificar a metodologia da linguagem UML.

4.1 Introdução

UML quer dizer Unified Modeling Language (Linguagem de Modelagem Unificada) e é uma linguagem que pode ser usada para descrever coisas do mundo real. É uma linguagem padrão para a elaboração da estrutura de projeto de software. A UML é utilizada para a modelagem de sistema, abrangendo todas as visões necessárias ao desenvolvimento desse sistema, independentemente da linguagem de programação a ser utilizada na implementação da solução do problema.

A UML é uma linguagem constituída de elementos gráficos (visuais), e a partir desses elementos podem-se construir diagramas que vão representar as diversas perspectivas para a construção de um sistema orientado a objeto.

- Sintaxe: cada elemento gráfico possui uma forma predeterminada de ser desenhado.
- Semântica: cada elemento gráfico possui um significado e é utilizado para algum fim.

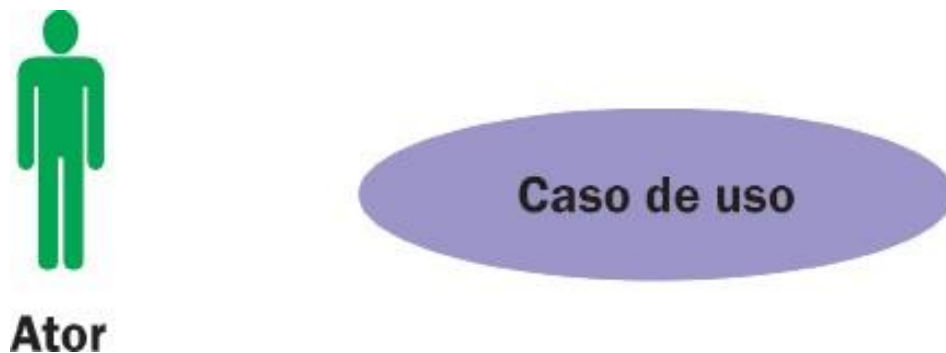


Figura 9. Elementos gráficos da UML.

A utilização da UML como linguagem de modelagem, em um processo de desenvolvimento, envolve a criação de diversos documentos.

- Estes documentos podem ser textuais ou gráficos.
- Estes documentos são denominados artefatos de software.
- São os artefatos que compõem as visões do sistema.
- Os artefatos gráficos produzidos durante o desenvolvimento de um sistema de software são definidos através da utilização dos diagramas da UML.

4.1.1 As 5 visões da UML

Segundo Bezerra (2011), a modelagem da arquitetura de um sistema abrange como o sistema será construído.

- **Visão de caso de uso:** descreve o sistema de um ponto de vista externo como um conjunto de interações entre o sistema e os agentes externos ao sistema.
- **Visão de projeto:** enfatiza as características do sistema que dão suporte às funcionalidades visíveis do sistema.
- **Visão de implementação:** abrange o gerenciamento de versões do sistema, construídas através do agrupamento de módulos e subsistemas.

- **Visão de implantação:** corresponde à distribuição física do sistema em seus subsistemas e à conexão entre essas partes.
- **Visão de processo:** esta visão enfatiza as características de concorrência (paralelismo), sincronização e desempenho do sistema.



Figura 9. Elementos gráficos da UML.

4.2 Ciclos iterativos

Uma vez feito o levantamento de requisitos, o próximo passo é organizá-los de forma a abordá-los nos ciclos iterativos, que se baseiam em casos de uso. Cada ciclo iterativo cumpre as fases de análise, projeto e programação.

O modelo iterativo incremental envolve a participação do usuário nas atividades de desenvolvimento do sistema, o que diminui a possibilidade de interpretações erradas em relação aos requisitos levantados.

- **Iterativo:** o sistema de software é desenvolvido em vários passos similares.
- **Incremental:** em cada passo, o sistema é estendido com mais funcionalidades.

4.3 Caso de uso

Segundo Jacobson, caso de uso é um documento narrativo que descreve a sequência de eventos de um ator que usa um sistema para completar um processo.

Caso de uso é uma técnica de modelagem usada para descrever o que um novo sistema deve fazer.

- É construído através de um processo iterativo em que as discussões entre o cliente e os analistas conduzem a uma especificação do sistema no qual todos estão de acordo com o entendimento do projeto.
- Descreve as operações que o sistema deve executar para cada usuário.
- Apresenta uma lista completa das interações entre um usuário e o sistema para cumprir uma tarefa do início até o fim.

Casos de uso estão associados a um conjunto de requisitos funcionais do sistema e aos atores envolvidos. O objetivo é levantar informações sobre como o sistema interage com os usuários.

- Descrever os requisitos funcionais do sistema.
- Fornecer uma descrição clara do que o sistema deve fazer.
- Permitir a descoberta dos requisitos funcionais das classes e operações do sistema.

O(a) analista deve descrever “o que” acontece entre o usuário e o sistema, sem se preocupar em informar “como” as interações vão ocorrer, nem com qual interface comunicará usuário e sistema.

A narrativa do caso de uso deve enfatizar “o que” um processo faz e não “como” o processo é efetuado. Deve descrever os principais fluxos de ações do sistema e as sequências das possíveis exceções.

Um caso de uso tem passos obrigatórios que envolvem a troca de informações dos atores para o sistema e do sistema para os atores, e a falta deles faz com que o caso de uso esteja incorretamente descrito.

No exemplo da aula anterior, na especificação dos requisitos de um sistema de biblioteca, o fluxo principal do caso de uso de “emprestar livro” deve conter obrigatoriamente os passos que indicam o momento nos quais o atendente (usuário) identifica o cliente e o livro no sistema.

Por que esses passos são obrigatórios?

Porque sem essas informações o sistema de biblioteca seria incapaz de registrar corretamente um empréstimo de livro. Não teria razão nenhuma para um sistema, registrar um empréstimo de livro sem indicar quem é o cliente que emprestou tal material.

A ausência de fluxo de interação que identifica essa troca de informação entre o ator e sistema e sistema e ator deixa os casos de uso sem sentido.

Depois de descrever o fluxo principal do caso de uso, podemos analisar o que poderia dar errado em cada um dos passos identificando os fluxos alternativos (exceções).

Uma exceção é um evento capaz de impedir o prosseguimento do caso de uso que não é tratado corretamente. A exceção não impede que o processo seja iniciado, mas impede sua conclusão.

No exemplo do sistema de biblioteca, o fato de o cliente não possuir cadastro válido não o impede de solicitar empréstimo de livro. Porém, a conclusão do caso de uso vai depender se ele tem uma identificação. Então, o processo foi iniciado, mas não pôde ser concluído. Cada exceção corresponde a uma ramificação do fluxo principal, e deve ser tratada para que o processo possa começar e terminar corretamente.

Ex.: Caso de uso: emprestar livro

Caso de uso: emprestar livro	
Fluxo principal	Exceções
1.O cliente seleciona os livros na biblioteca que deseja emprestar. 2.O cliente informa ao funcionário seu nome e os livros que serão emprestados. 3.O funcionário informa o nome do cliente no sistema e inicia o processo de “emprestar livro”. 4. O funcionário registra cada livro no sistema. 5.O funcionário finaliza o processo de “emprestar livro”, informando a data de devolução. 6. O Cliente deixa a biblioteca com os livros.	1. O cliente não possui cadastro. 1. O cliente deve informar seus dados para cadastro. 2. O funcionário registra o cadastro. 3. Retorna ao fluxo principal 3.

Figura 11. Exemplo de fluxo principal e exceções para um caso de uso.

4.4 Componentes de um modelo de caso de uso

- Ator – papel que estimula/solicita ações/ eventos do sistema e recebe reações. Pode participar de vários casos de uso.
- Caso de uso – documento narrativo que descreve a sequência de eventos feitos por um ator no uso do sistema.
- Sistema – sistema a ser modelado.

Relacionamento

O relacionamento representa uma interação existente entre o sistema e seus componentes com o ambiente externo. Ocorre de quatro maneiras diferentes:

- **Comunicação:** representa troca de mensagens.
- **Inclusão:** representa um caso que, ao ser executado, deve executar um outro caso de uso também.
- **Extensão:** representa um caso que, ao ser executado, pode ou não executar um ou mais casos de uso também.
- **Generalização:** representa um caso de uso ou um ator que herda características de outro caso de uso ou ator.



4.5 Documentação do modelo de caso de uso

O modelo de casos de uso é composto por:

- **diagrama** de casos de uso;
- documentação dos **atores**; e
- documentação dos **casos de uso**.

4.5.1 Diagrama de casos de uso

O diagrama de caso de uso serve para dar suporte à parte escrita do mode-

lo. Basicamente, um diagrama de caso de uso apresenta dois elementos: os atores e os casos de uso.

Dependendo do grau de complexidade do sistema, o diagrama pode ser apresentado:

- Exibindo um caso de uso de seus relacionamentos.
- Exibindo todos os casos de uso para um ator.
- Exibindo todos os casos de uso a serem implementados em um ciclo de desenvolvimento.

Ex.: diagrama que representa a relação entre ator e o caso de uso.

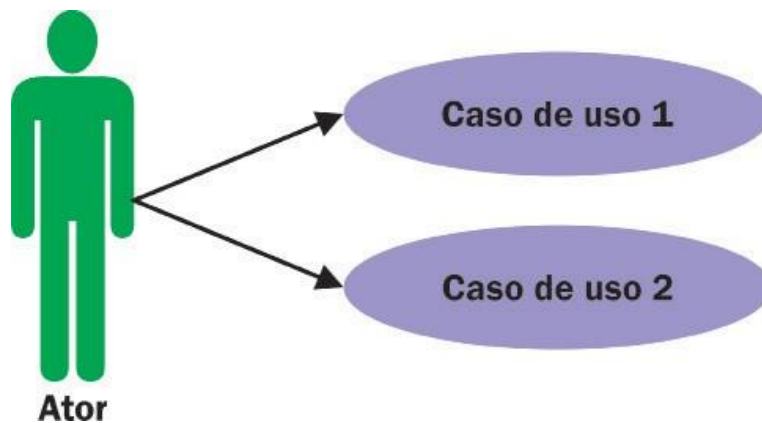


Figura 12. Representação básica da UML.

A utilização de setas nas relações entre atores e casos de uso indica qual ator ativa o caso de uso.

Ex.: relacionamentos em casos de uso.

Existem três tipos de relações entre casos de uso (inclusão, extensão e generalização):

- **Relacionamento de inclusão**

Um relacionamento de inclusão é uma relação através da qual um caso de uso insere em seu interior um outro caso de uso, que isoladamente não teria sentido. Ele é um integrante de um caso de uso maior e completo.

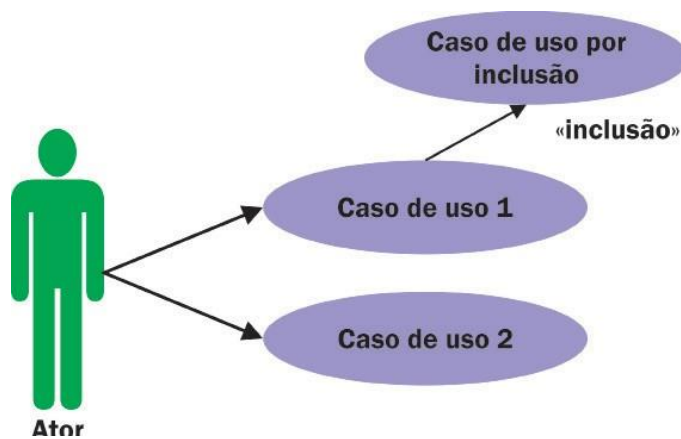


Figura 13. Representação da UML de relacionamento de inclusão em um caso de uso.

- **Relacionamento de extensão**

Um relacionamento de extensão é uma relação entre dois casos de uso na qual um caso de uso maior é estendido por um caso de uso menor. A extensão inclui algum serviço ou alguma funcionalidade que ocorrem em dada situação.

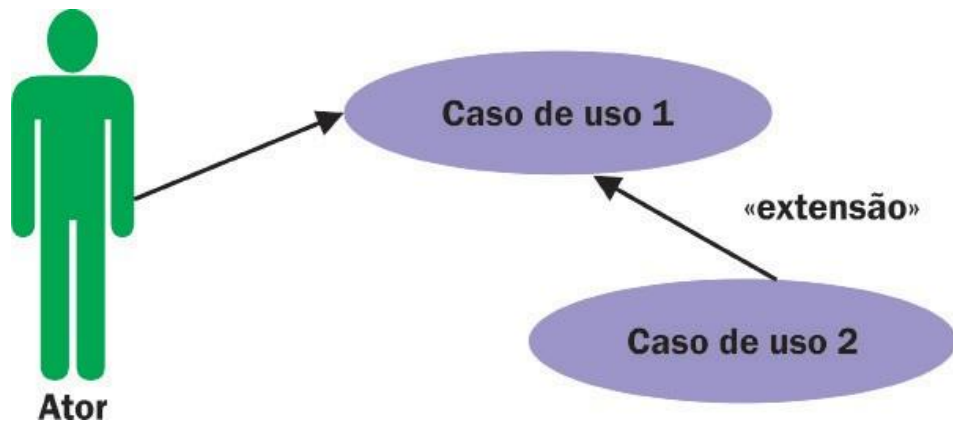


Figura 14. Representação da UML de relacionamento de extensão em um caso de uso.

- **Relacionamento de generalização**

Um relacionamento de generalização é uma relação entre um caso de uso mais geral e um caso de uso mais específico. Podemos dizer que o caso de uso mais geral é uma generalização (abstração) do caso de uso mais específico. A representação gráfica da UML para relacionamento de generalização é uma reta que une os dois casos de uso. Essa reta possui um triângulo na extremidade apontando para o caso de uso mais geral.

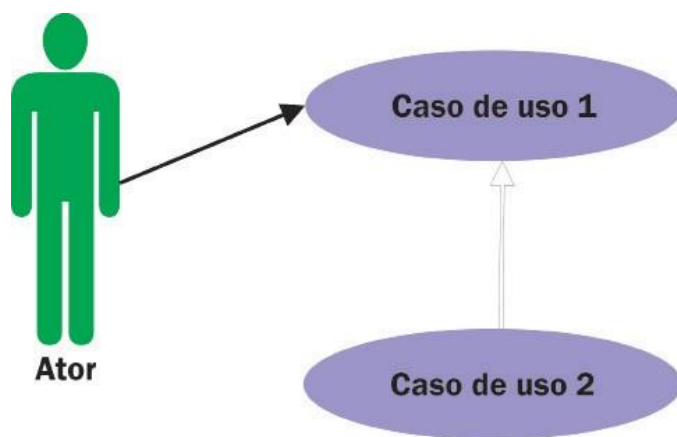


Figura 15. Representação da UML de relacionamento de generalização entre casos de uso.

2. Documentação dos atores

Deve conter uma breve descrição para cada ator.

3. Documentação dos casos de uso

Um caso de uso representa como os atores interagem com o sistema e como o sistema responde. É iniciado por um ator ou por outro caso de uso.

Para cada caso de uso deverá existir:

- Um nome composto por um verbo seguido por um substantivo.
- Um identificador que permita identificá-lo de forma única.
- Uma pequena descrição do caso de uso.
- O nome do ator primário. Isto é, aquele que inicia o caso de uso.
- Os nomes dos atores secundários. Isto é, os demais atores participantes do caso de uso, se existirem.
- Fluxo que representa a sequência de passos principais do caso de uso, o qual é chamado de fluxo principal.
- Fluxo principal com alternativas independentes, o qual descreve a sequência de eventos alternativo(s) para um caso de uso.
- Fluxo principal com alternativas exclusivas entre si: descreve uma sequência de eventos alternativo(s) para um caso de uso, em que apenas um

deles pode ocorrer por vez.

- Fluxo de exceção: descreve a sequência de eventos quando algo inesperado acontece na interação entre ator e caso de uso.

Vou exemplificar a modelagem de um caso de uso: sistema de controle de biblioteca.

Descrição do problema

O objetivo de um sistema para uma biblioteca é automatizar o controle de empréstimo e devolução de livros.

O sistema deve manter o controle dos cadastros de usuários da biblioteca e o acervo de livros, os empréstimos e devolução de livros pelos usuários da biblioteca.

O usuário/cliente da biblioteca poderá acessar o sistema pela internet e fazer a reserva e o cancelamento da reserva de livros.

Se o empréstimo do livro não for devolvido dentro do prazo estipulado, será cobrada uma multa.

Para identificarmos os atores que irão interagir com o sistema, podemos responder algumas perguntas, como:

- Qual o ator que irá operar o sistema? R: operador do sistema.
- Qual o ator que fará a manutenção do sistema? R: operador do sistema.
- Qual o ator que receberá as informações do sistema? R: operador do sistema, usuário da biblioteca.
- Qual o ator que fornecerá as informações para o sistema? R: operador do sistema, usuário da biblioteca.

Portanto, os atores identificados para interagir com o sistema são:

- Operador do sistema (bibliotecário) – responsável pelas operações de controle de empréstimo e devolução de livros, controle do cadastro de

usuários da biblioteca e controle do acervo dos livros.

- Usuário da biblioteca (cliente) – responsável por fornecer as informações cadastrais, solicitar reserva de livro e cancelar reserva de livro.

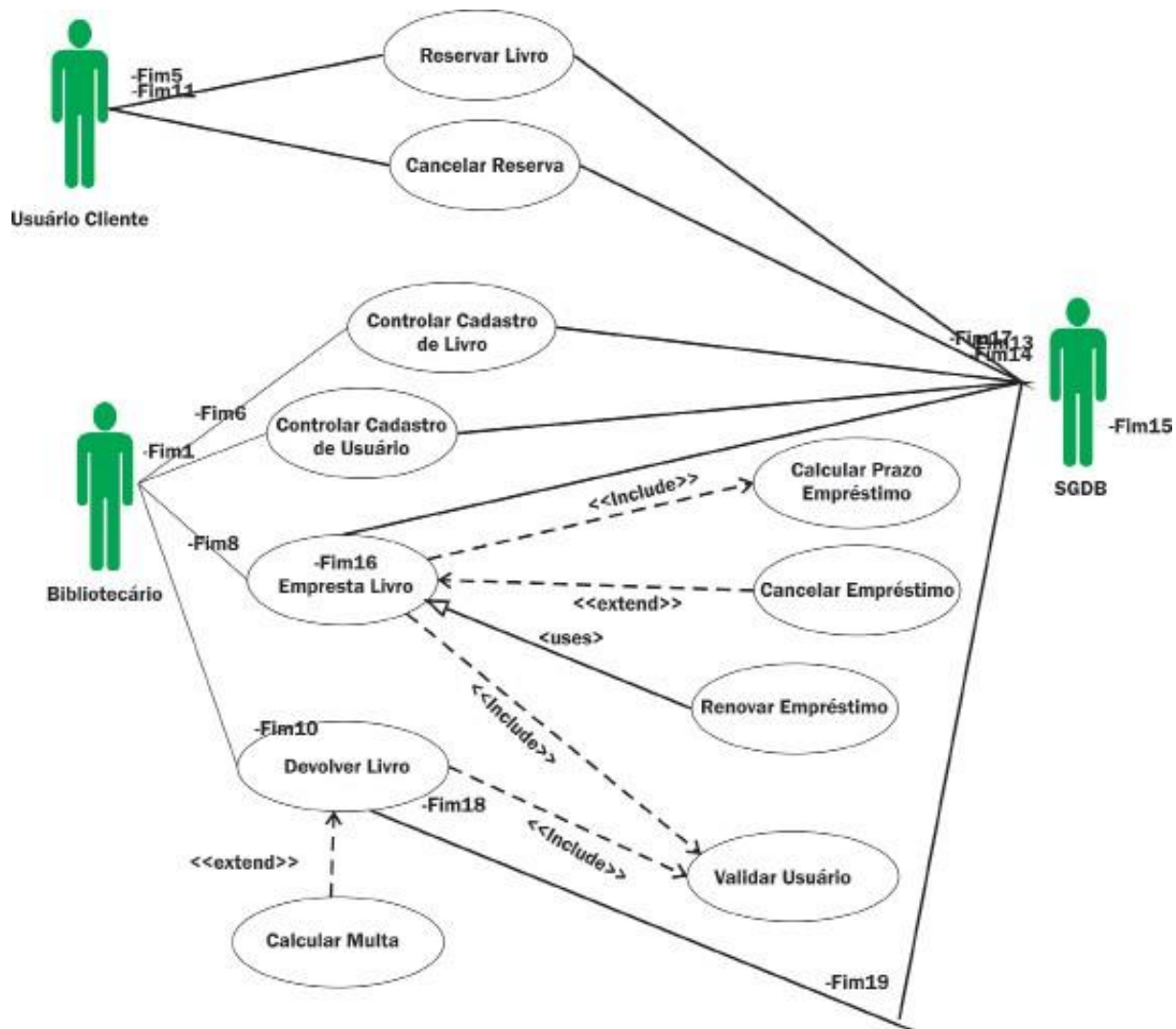


Figura 16. Representação gráfica do caso de uso biblioteca.

Vou agora descrever cada caso de uso.

- O caso de uso deve receber um nome único.
- Os atores devem ser declarados.
- A declaração do caso de uso deve descrever o que ele realmente faz.

Caso de uso: controlar cadastro de livro
Objetivo: inserir, consultar, alterar, excluir dados de livro no sistema ou cancelar.
Ator principal: usuário (bibliotecário).
Ator secundário: SGDB (Sistema Gerenciador de Banco de Dados, onde ficarão armazenadas as informações).
Precondição: usuário (bibliotecário) ser identificado no sistema.
Fluxo Principal:
1. O sistema exibe a tela inicial para incluir as informações de cadastro do novo livro.
2. O sistema valida as informações inseridas pelo usuário (bibliotecário). Emite mensagem de “Dados Válidos”.
3. Caso de uso encerrado.
Fluxo Alternativo:
FA1. Consultar cadastro de livros
O sistema exibe a tela de consultar para informar o nome do livro.
O sistema exibe a lista de consulta de acordo com o filtro solicitado pelo usuário e detalha a informação.
O sistema solicita finalização da consulta (OK?).
O sistema retorna ao passo 3 do fluxo principal.
FA2. Alterar cadastro de livros
O sistema exibe a tela de alterar para informar o nome do livro.
O sistema mostra as informações sobre o livro especificado no filtro.
O usuário altera as informações desejadas.
O sistema valida as informações alteradas pelo usuário.
O sistema solicita finalização da alteração (OK?).
O sistema retorna ao passo 3 do fluxo principal.
FA3. Excluir livro do cadastro
O sistema exibe a tela de exclusão para informar o nome do livro.
O sistema mostra as informações sobre o livro especificado no filtro.
O sistema solicita confirmação de exclusão dos dados no cadastro de livros.
O usuário confirma a solicitação de exclusão.
O sistema valida opção pelo usuário.
O sistema retorna ao passo 3 do fluxo principal.
FA4. Cancelar operação
O sistema exibe a opção de cancelar. O usuário seleciona operação cancelar.
O sistema retorna ao passo 3 do fluxo principal.



Caso de uso: controlar cadastro de usuário/cliente
Objetivo: inserir, consultar, alterar, excluir dados de cliente no sistema ou cancelar.
Ator principal: usuário (bibliotecário).
Ator secundário: SGDB (Sistema Gerenciador de Banco de Dados, onde ficarão armazenadas as informações).
Precondição: usuário (bibliotecário) ser identificado no sistema.
Fluxo Principal:
1. O sistema exibe a tela inicial para incluir as informações cadastrais do novo cliente.
2. O sistema valida as informações inseridas pelo usuário (bibliotecário). Emite mensagem de "Dados Válidos".
3. Caso de uso encerrado.
Fluxo Alternativo:
FA1. Consultar cadastro de cliente
O sistema exibe a tela de consulta para informar a identificação do cliente.
O sistema exibe a lista de consulta de acordo com o filtro solicitado pelo usuário e detalha a informação.
O sistema solicita finalização da consulta (OK?).
O sistema retorna ao passo 3 do fluxo principal.
FA2. Alterar cadastro de cliente
O sistema exibe a tela de alterar para informar a identificação do cliente.
O sistema mostra as informações sobre o cliente especificado no filtro.
O usuário (bibliotecário) altera as informações desejadas.
O sistema valida as informações alteradas pelo usuário (bibliotecário).
O sistema retorna ao passo 3 do fluxo principal.
FA3. Excluir cliente do cadastro
Opção excluir: o sistema exibe a tela de exclusão para informar a identificação do cliente.
O sistema mostra as informações sobre o cliente especificado no filtro.
O sistema solicita confirmação de exclusão dos dados no cadastro de cliente.
O usuário (bibliotecário) confirma a solicitação de exclusão.
O sistema valida opção pelo usuário (bibliotecário).
O sistema retorna ao passo 3 do fluxo principal.
FA4. Cancelar operação
O sistema exibe opção de cancelar. O usuário (bibliotecário) seleciona operação cancelar.
O sistema retorna ao passo 3 do fluxo principal.

Caso de uso: emprestar livro
Objetivo: possibilitar que o cliente possa emprestar um livro.
Ator principal: usuário (bibliotecário).
Ator secundário: SGDB (Sistema Gerenciador de Banco de Dados, onde ficarão armazenadas as informações).
Precondição: usuário (bibliotecário) ser identificado no sistema.
Fluxo Principal:
1. O sistema exibe a tela inicial e usuário (bibliotecário) informa a identificação do cliente e do livro a ser emprestado.
2. O Sistema valida o cliente mostrando informações de cadastro.
3. O sistema verifica se existe uma reserva do livro para o cliente identificado.
4. O sistema verifica a disponibilidade de exemplar para o livro desejado.
5. O sistema registra o empréstimo e calcula prazo de devolução do livro.
6. O sistema emite recibo contendo as informações do empréstimo.
7. Caso de uso encerrado.
Fluxo Alternativo:
FA1. Reserva de livro para outro cliente
O sistema emite mensagem de “Livro Reservado” para o usuário (bibliotecário).
O sistema retorna ao passo 7 do fluxo principal.
FA2. Reserva do livro pelo cliente
O sistema dá baixa na reserva do livro pelo cliente.
O sistema retorna ao passo 4 do fluxo principal.
FA3. Renovar empréstimo
O sistema verifica o prazo de entrega e emite mensagem de “Renovar Empréstimo” para o usuário (bibliotecário).
O sistema retorna ao passo 5 do fluxo principal.
FA4. Cancelar empréstimo
O sistema exibe a opção cancelar. O usuário (bibliotecário) seleciona operação cancelar empréstimo.
O sistema retorna ao passo 7 do fluxo principal.

Caso de uso: devolver livro
Objetivo: possibilitar que o cliente devolva um livro emprestado.
Ator principal: usuário (bibliotecário).
Ator secundário: SGDB (Sistema Gerenciador de Banco de Dados, onde ficarão armazenadas as informações).
Precondição: usuário (bibliotecário) ser identificado no sistema.
Fluxo Principal:
1. O sistema exibe a tela inicial e usuário (bibliotecário) informa a identificação do cliente e do livro a ser devolvido.
2. O Sistema valida o cliente mostrando informações de cadastro.
3. O sistema identifica registro de empréstimo.
4. O usuário (bibliotecário) confirma devolução do livro.
5. O sistema registra a devolução e verifica se houve atraso no prazo de devolução do livro.
6. O sistema emite mensagem de “Devolução no Prazo”.
7. Caso de uso encerrado.
Fluxo Alternativo:
FA1. Devolução em atraso.
O sistema calcula o número de dias em atraso e o valor da multa.
O sistema emite recibo de pagamento no valor da multa.
O usuário (bibliotecário) registra o pagamento da multa no sistema.
O sistema retorna ao passo 7 do fluxo principal.



Vamos determinar algumas regras básicas que definirão a política de controle do sistema de biblioteca.

Regras de negócio
RN01. No sistema de controle de biblioteca, um cliente não pode ter mais de uma reserva de livro em seu nome.
RN02. Depois de feita a reserva de um livro, o cliente tem prazo de 48 horas para a retirada do livro na biblioteca.
RN03. Um cliente só pode ter dois livros em seu poder ao mesmo tempo.
RN04. O valor da multa a ser aplicada é de R\$ 2,00 por dia de atraso.

Resumo

A UML é uma linguagem de modelagem padronizada que permite visualizar e entender o relacionamento entre os objetos definidos.

A identificação dos casos de uso de um sistema é, em muitas vezes, um processo intuitivo, em que a utilização da UML permite especificar as funcionalidades do sistema e a relação existente entre os atores através do diagrama. E através do documento de caso de uso, no qual é declarado o fluxo principal e os fluxos alternativos, que será descrito como cada caso de uso deve funcionar.

A declaração das regras de negócio vai definir a política adotada para o tratamento das particularidades das funcionalidades do sistema.



Aula 5. Modelagem de classe de objeto

Objetivo:

- construir um diagrama de classes através da linguagem UML.

E então, o que é o diagrama de classe da UML?

5.1 Introdução

O diagrama de classe da UML é uma ferramenta utilizada para a construção do modelo de classes de objetos para a documentação de software.

Identificamos o levantamento das classes do sistema pelos casos de uso definidos no projeto do sistema. Uma classe descreve um grupo de objetos com propriedades (atributos), comportamento (operações) e os relacionamentos, e pode ser empregada em mais de um caso de uso.

O modelo de caso de uso representa a visão do sistema do ponto de vista externo, no qual os atores visualizam as solicitações de informação, requisitos de cálculo etc. Porém, os objetos precisam trocar informações, ter uma interação uns com os outros para produzir os resultados que o sistema requer.

Como vimos na segunda aula deste caderno, a representação gráfica de uma classe é composta de 3 (três) partes: o nome da classe, declaração dos atributos e a declaração das operações.

NomeClasse
Atributos
Operações ()

Figura 17. Representação gráfica de uma classe.

2. Identificação de classes

O primeiro passo para identificar as classes que vão fazer parte da modelagem do sistema é fazer uma leitura do levantamento de requisitos para determinar os substantivos que possam ser classificados como os possíveis candidatos as classes. Nessa leitura serão verificados quais os eventos que fazem parte do requisito, quem (clientes, alunos etc.) interage direta ou indiretamente com o sistema, locais que fazem parte do problema, como departamentos, setores, projetos que são informações relevantes para a modelagem do sistema. Em uma classe precisam ser identificados pelo menos dois ou mais atributos.

3. Identificação de atributos

Os atributos representam as propriedades de uma entidade ou de um relacionamento que está associado a um conjunto de valores válidos para o atributo.

Graficamente é representado por uma elipse.

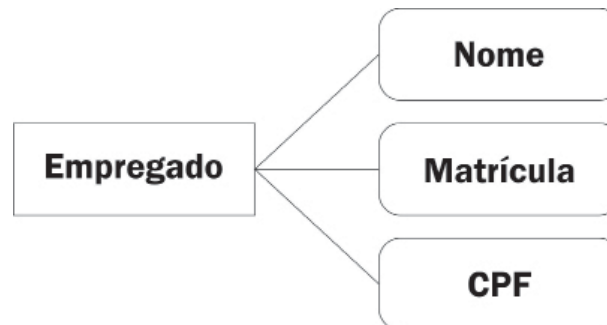


Figura 18. Representação gráfica de uma entidade e seus atributos.

Os relacionamentos também podem ter atributos específicos que identifiquem relações.

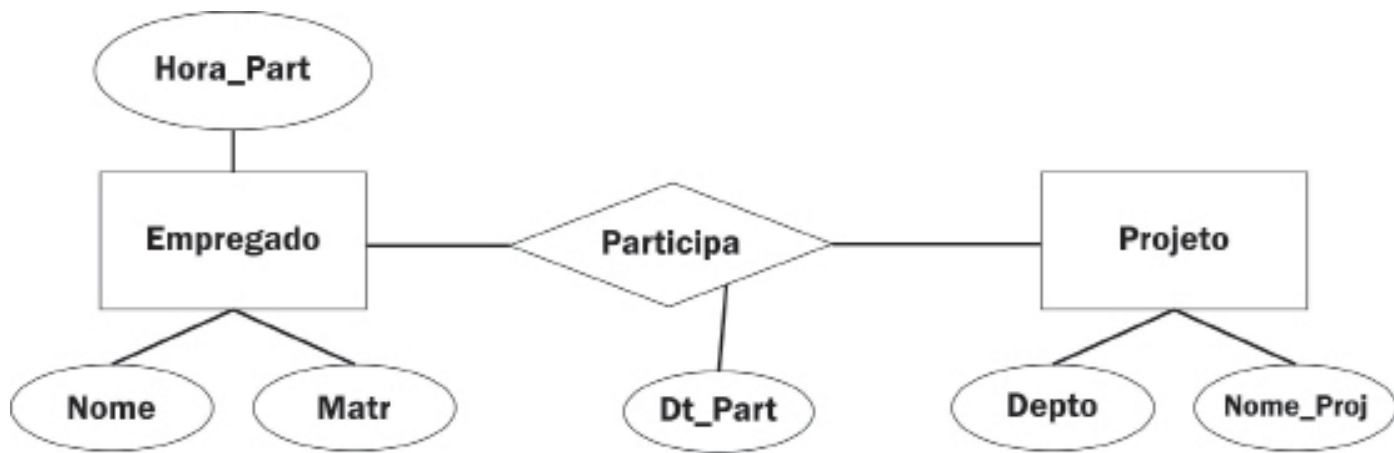


Figura 19. Representação gráfica de atributos de relacionamento.

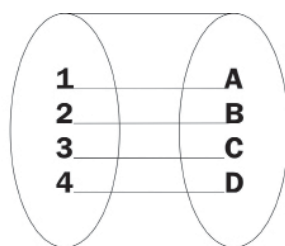
5.4 Identificação de relacionamentos

O que é relacionamento?

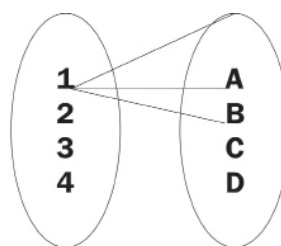
Relacionamento é a associação entre os objetos durante a execução do sistema. Cada associação tem uma cardinalidade que indica quantos objetos podem participar de um relacionamento.

Definimos o relacionamento entre as entidades representado em um diagrama entidade/relacionamento através de uma linha unindo as entidades com um losango ao meio, com o nome do relacionamento (um verbo).

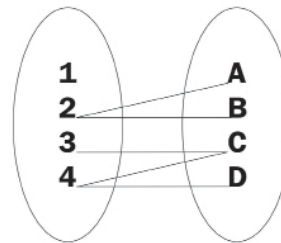
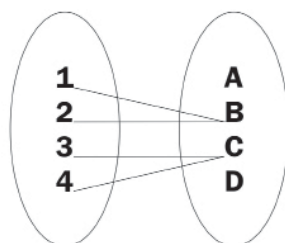
- Vamos ver uma das características dos relacionamentos, que é a cardinalidade. A cardinalidade representa o número de vezes que cada elemento de uma entidade pode participar da relação.



Relacionamento um-para-um (1:1)



Relacionamento um-para-muitos (1:N)



Vamos exemplificar para entender melhor.

Ex1.: Cardinalidade (N:N)



Podemos dizer que cada entidade de pacientes se relaciona por meio de consultar com várias entidades de médicos. Da mesma forma que uma entidade de médico se relaciona por meio de consultar com várias entidades de pacientes.

Consideramos nesse exemplo que muitos médicos podem consultar muitos pacientes e que muitos pacientes podem se consultar com muitos médicos.

Ex2.: Cardinalidade (1:1)



A entidade empregado se relaciona por meio do relacionamento coordenar com a entidade departamento.

Podemos dizer que um único empregado exerce a função de coordenar um único departamento.

Ex3.: Cardinalidade (1:N)



No exemplo acima, o 1 anotado na aresta que liga alocar e departamento, indica que cada entidade de departamento pode estar relacionada por meio de alocar com no máximo uma entidade de departamento. E o N anotado na aresta que liga alocar e empregado, indica que cada entidade de departamento pode estar relacionada por meio de alocar com um número qualquer de entidades de empregado.

Podemos dizer que um departamento pode alocar vários empregados, ou vários empregados são alocados por um departamento.

5.5 Relacionamento entre classes

Tipos de relacionamento entre classes e objetos:

- Relacionamento por associação
- Relacionamento por agregação
- Relacionamento por composição
- Relacionamento por generalização (herança)

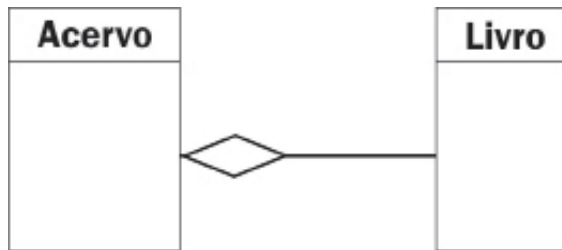
5.5.1 Relacionamento por associação

O relacionamento por associação representa as ligações entre os objetos de duas classes.



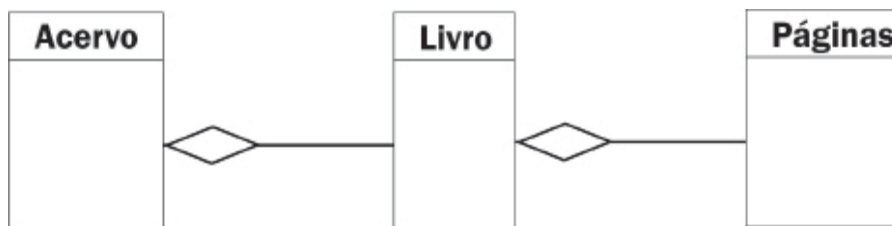
5.5.2 Relacionamento por agregação

O relacionamento por agregação é uma associação na qual o elemento associado é parte do elemento principal. É representado por uma haste com o símbolo de um diamante junto à classe agregadora.

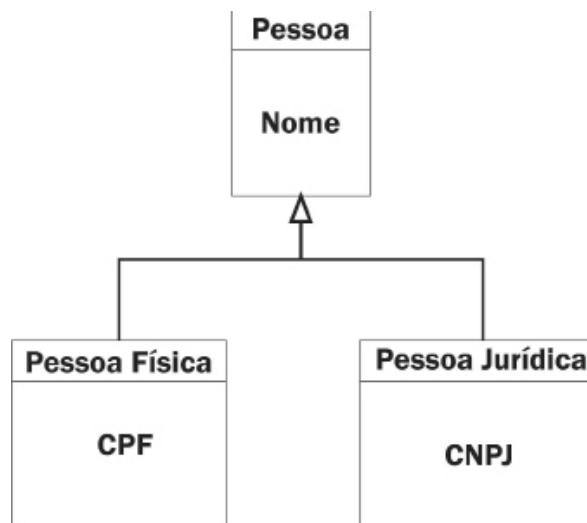


5.5.3 Relacionamento por composição

O relacionamento por composição é um tipo de agregação em que a parte indicada deve necessariamente existir. Não podemos pensar no objeto da classe principal sem os objetos que o compõem.



5.5.4 Relacionamento por generalização (herança)



Definimos que as classes `Pessoa Física` e `Pessoa Jurídica` herdam as características da classe `Pessoa`.

5.6 Diagrama de classe

Vamos considerar o sistema de controle de biblioteca, definido na quarta aula para identificarmos as classes do projeto. Nesta fase de análise, e co-

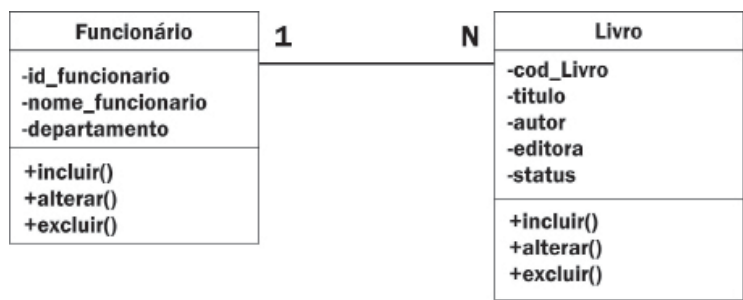
analisando os casos de uso do sistema, podemos desenhar o diagrama de classe que irá representar o agrupamento dos objetos e seus relacionamentos.

Consideremos:

- **Casos de uso principal:** reservar livro, cancelar livro, controlar cadastro de livro, controlar cadastrar usuários, emprestar livro, devolver livro.
- **Objetos:** livro, funcionário, cliente, empréstimo, devolução, reserva, pagamento, prazos.
 - o que o sistema vai controlar?
 - como o sistema vai controlar?
 - quem vai utilizar o sistema?
- **Classes:** livro, funcionário, cliente, empréstimo, devolução, reserva.
- **Operações:** incluir, alterar, excluir, consultar.

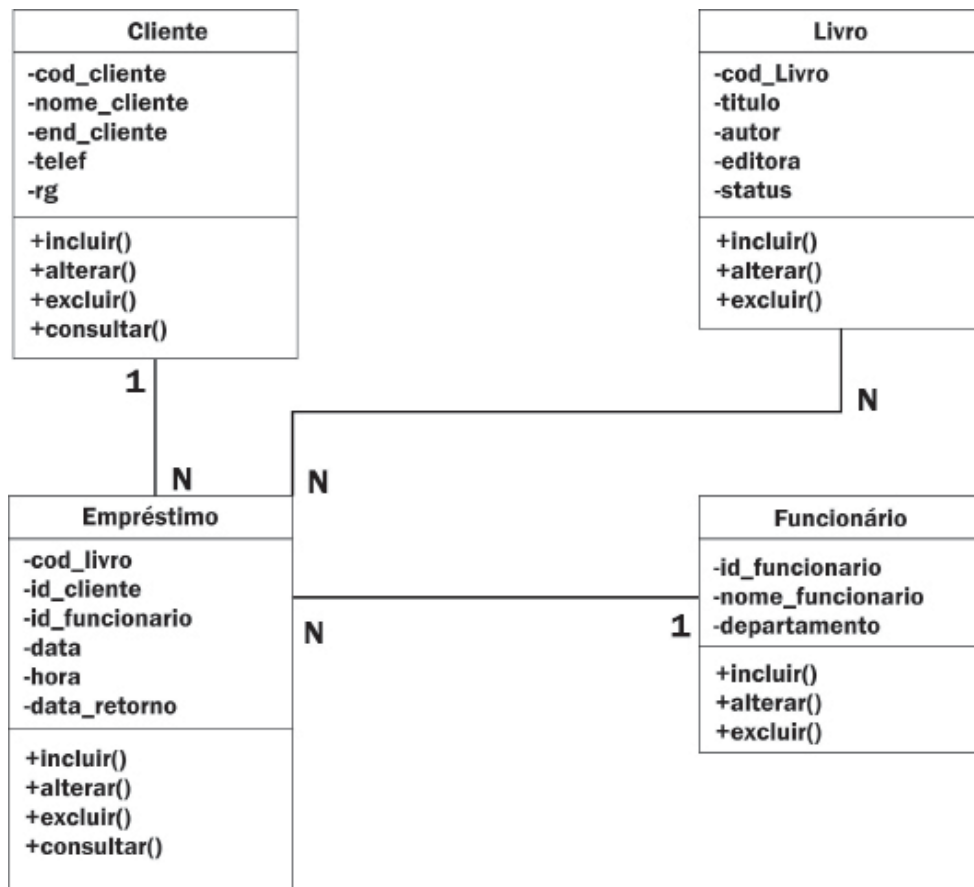
Vou exemplificar um caso de uso para melhor entender o diagrama de classes

Caso de uso 1: controlar cadastro de livro



Podemos dizer que um funcionário opera as funções de controle de vários livros.

Caso de uso 2: controlar empréstimo de livro



Podemos dizer que um funcionário controla um ou mais de um empréstimo de livro solicitado por um cliente.

Resumo

Nesta aula você observou que o diagrama de classe da UML é uma ferramenta utilizada para a construção do modelo de classes de objetos para a documentação de software. Você teve a oportunidade de aprender que o primeiro passo para identificar as classes que vão fazer parte da modelagem do sistema é fazer uma leitura do levantamento de requisitos para determinar os substantivos que possam ser classificados como os possíveis candidatos as classes. Por fim, você viu que relacionamento é uma associação entre os objetos durante a execução do sistema e que cada associação tem uma cardinalidade que indica quantos objetos podem participar de um relacionamento.